MCP PRODUCTION · SPEC REVISION 2026-07-28

The healthy replica that still fails a tool call

The incident sheet from lesson 8.1: nine observations from a controlled run, what each one establishes, five questions and the answer key.

Everything you need for lesson 8.1 is on this sheet: no lab to run, no earlier lesson. The observations come from a controlled local run on 2026-09-10, not from a real service's incident.

The evidence you have

A client calls the lookup tool through a gateway. The same replica can answer its health endpoints while the lookup dependency refuses a request.

OBSERVATION	RECORDED OUTCOME	WHAT IT ESTABLISHES
Replica `/livez`	HTTP 200	The health handler answered
Replica `/readyz`	HTTP 200	The configured admission probe passed
Lookup call	HTTP 200, `isError: true`	A tool failure was delivered successfully
Lookup result text	`{"error":"dependency_unavailable"}`	The local dependency adapter reported refusal
Independent export call during this fault	HTTP 200	Another application path still accepted work
Dependency restored	Lookup `isError: false`	The injected failure ended after restoration
Shared application store exclusively locked	`/readyz` HTTP 503	The required store check failed
Same locked-store case	`/livez` HTTP 200	The process health handler still answered
Store lock released	Readiness and lookup HTTP 200	The local paths recovered

The readiness policy checks whether the replica is draining and whether it can obtain a bounded write lock in its application store. It deliberately excludes the lookup dependency because export work does not use it. All application paths do need the store, including grant validation. A probe samples one instant; it does not promise that the next request will succeed.

The transport status and tool outcome belong to different layers. HTTP 200 alone does not establish that lookup succeeded. The failure result here is a complete response containing `isError: true`, not a failed health endpoint.

Work through the diagnosis

1. Would restarting every replica repair the refused dependency? State what evidence is missing before choosing that action.

2. If readiness began checking this optional dependency, which still-useful path would lose admission during the same incident?
3. When the shared store is locked, why does this policy withdraw readiness?
4. Which observation would distinguish a scope denial from this tool failure?
5. If a write times out at the gateway, does that establish whether it committed?

Answer key

1. The observations establish a local dependency fault. They do not establish that replica restart repairs that dependency. Inspect the dependency path and its recovery evidence before broadening the intervention.
2. Export admission would be withdrawn too. That may be an intentional service policy, but it trades away a path the evidence shows was still useful.
3. The store supports authorization and application state for the admitted service, so a failed bounded access check affects the shared request path.
4. Inspect the access decision and correlated response. A permission denial should be identified separately from a successfully authorized call whose dependency fails; do not infer either from a generic failure count.
5. No. A gateway timeout ends its observation. A write may commit later; inspect durable operation identity and state before deciding whether to repeat it.

For your own service, write down the failed operation, caller context, request correlation, replica, version, elapsed time and dependency outcome. Keep secrets and raw personal payloads out of diagnostic records. Label what each observation proves and list the next measurement that would separate remaining hypotheses.

Sources

Checked 2026-09-10:

- Probe purposes
- Request and result model

The table has every observation the exercise needs. The same faults are reproduced by tests 6 and 7 in `lab-8-operations/tests.py`, if you want to watch them happen.

> **Where this goes next.** Lab 8 puts the whole service behind a load balancer, breaks a dependency and a replica on purpose, and rehearses a release and its rollback. The course: runsclean.com/courses/mcp-production/